mode. But the example is not significant, because of the assumption that the provisioned throughput of 10 writes and 100 reads per second is running at 100% capacity 24/7. This is hardly true for any application. Most applications have large variations in read and write throughput. Therefore, in many cases, the opposite is correct: on-demand is more cost-efficient than provisioned. For example, we are operating a chatbot called marbot on AWS. When we switched from provisioned to on-demand capacity in December 2018, we reduced our monthly costs for DynamoDB by 90% (see http://mng.bz/DD69 for details).

As a rule of thumb, the more spikes in your traffic, the more likely that on-demand is the best option for you. Let's illustrate this with another example. Assume your application is used only during business hours from 9 a.m. to 5 p.m. The baseline throughput is 100 reads and 10 write per second. However, a batch job is running between 4 p.m. and 5 p.m., which requires 1,000 reads and 100 writes per second.

With provisioned capacity mode, you need to provision for the peak load. That's 1,000 reads and 10 writes per second. That's why the monthly costs for a table with provisioned capacity costs is $24.75 higher than when using on-demand capacity in this example, as shown here:

```
// Provisioned capacity mode
$0.00065 for 1 writes/sec * 100 * 24 hours * 30 days = $46.80 per month
$0.00013 for 2 reads/sec * 500 * 24 hours * 30 days = $46.80 per month

Read/Writes with provisioned capacity mode per month: $93.60

// On-demand Capacity Mode
(7 hours * 60 min * 60 sec * 10 write) + (1 hour * 60 min * 60 sec * 100 writes) =
612,000 writes/day
(7 hours * 60 min * 60 sec * 100 reads) + (1 hour * 60 min * 60 sec * 1000 reads) =
6,120,000 reads/day
```